

Day 1 — PLC Foundations

Concept Summary, Student Tasks, and Instructor Solutions

Concept Summary

PLC Basics. A PLC is a purpose-built industrial computer running one control program in a deterministic, cyclic loop: read inputs → solve logic → write outputs. It replaces relay panels because logic changes happen in software (edit + download), not by re-wiring physical relays.

PLC Hardware. Power supply converts mains AC to 24V DC (safe voltage, industry-standard, low drop tolerance issues). Input modules read voltage as a logic 1/0 based on thresholds. Output modules come in three types: Relay (AC/DC, slower, mechanical wear), Transistor (DC only, fast, no wear), Triac (AC only, fast, no wear). Module type must match the load's electrical requirements before wiring.

PLC Scan Cycle. Three phases repeat every cycle: (1) INPUT SCAN freezes all inputs into the Input Image Table, (2) LOGIC SOLVE executes the program against that frozen table, (3) OUTPUT UPDATE writes results to physical terminals. Any output that 'didn't update when expected' is explained by scan timing — never by the PLC 'not noticing.' A pulse shorter than one scan time can be missed entirely.

Inputs/Outputs + NO/NC Contacts. NO (Normally Open) contacts read 0 at rest, 1 when actuated. NC (Normally Closed) contacts read 1 at rest, 0 when actuated. E-stops are wired NC so any wiring fault (cut wire, unplugged connector) reads identically to 'button pressed' — the machine stops safely. Wiring an E-stop NO would let a real fault go undetected while the machine keeps running.

Ladder Logic Basics. Contacts (conditions) sit on the left of a rung, a coil (result) on the right. AND = series contacts. OR = parallel contacts (branches). NOT = an NC-style contact symbol. Structured Text (ST) equivalents: 'Coil := A AND B;', 'Coil := A OR B;', 'Coil := NOT A;'.

Tag Naming. Raw addresses (%I0.0, %Q0.1) carry no semantic meaning. Production code uses symbolic tags (Start_PB, Motor_Run) mapped to the same physical address, so the logic is readable by a future maintainer.

Motor Start/Stop with Seal-In. A momentary Start button must energize an output that stays on after release. This needs the output's own contact wired in parallel with Start (the 'seal-in'), combined with a Stop condition (wired NC) that can break the circuit regardless of Start's state. ST: Motor_Run := (Start_PB OR Motor_Run) AND NOT Stop_PB;

Tasks & Solutions

Organized by Day 1 concept block. Each task is followed immediately by its solution/answer key for instructor use.

Task 1 — Architecture Labeling (PLC Basics / Hardware)

Given a blank version of the Power Supply → CPU → I/O Modules diagram, label the signal direction arrows and the voltage present at each stage.

Solution / Answer Key

Power Supply --24V DC--> CPU <--backplane bus--> Input Modules <-- field wiring -- sensors/switches

CPU --backplane bus--> Output Modules --field wiring--> actuators/lamps/motors

Mains side: 100–240V AC in. I/O side: 24V DC throughout. Key point students must state: the physical wiring topology does not change when logic changes — only the program in memory changes.

Task 2 — Relay vs. PLC Scenario (PLC Basics)

"A packaging line needs a new interlock added." Write 2–3 sentences comparing what changes physically (relay panel) vs. what changes in software (PLC), using the terms *stored program* and *I/O image table*.

Solution / Answer Key

Model answer: In a relay panel, adding the interlock means physically re-wiring contacts across the rack — a job that can take hours and risks wiring errors. In a PLC, the CPU solves the interlock as part of its *stored program*, executed each scan against the frozen *I/O image table*; the physical field wiring to sensors and actuators stays untouched, and only the program in memory is edited and downloaded.

Task 3 — Output Module Selection (PLC Hardware)

For each load below, select Relay / Transistor / Triac and justify in one line: (a) 230V AC solenoid, (b) high-frequency PWM DC valve, (c) 24V DC indicator lamp switched 50,000 times/day.

Solution / Answer Key

(a) **Relay or Triac** — must switch AC; a transistor output physically cannot drive an AC load.

(b) **Transistor** — DC only, no mechanical wear, suited to high-frequency/PWM-style switching.

(c) **Transistor** — DC load with a high switching count exceeds a relay's mechanical cycle rating ($\sim 10^5$ – 10^6 cycles); solid-state avoids wear.

Task 4 — Trace the Scan (PLC Scan Cycle)

Given a single-rung program (one NO contact → one coil) and a timeline showing a button press relative to scan boundaries, mark which scan cycle the output actually turns on, and explain why it isn't instantaneous.

Solution / Answer Key

The output does not turn on during the same instant the button is pressed. It turns on at the OUTPUT UPDATE phase of the *first full scan cycle that begins after* the press — i.e., the next INPUT SCAN must capture the pressed state into the Input Image Table, then LOGIC SOLVE evaluates it, then OUTPUT UPDATE writes it out. If the press happens mid-scan, it is not seen until the following cycle's input scan.

Task 5 — Missed Pulse Diagnosis (PLC Scan Cycle)

Scenario: a fast proximity sensor pulse is not being detected by the PLC. Write the one-sentence diagnosis using scan time vs. pulse duration.

Solution / Answer Key

The pulse duration is shorter than the PLC's scan time, so the signal went true and false again between two INPUT SCAN phases and was never latched into the Input Image Table — the PLC did not "miss" it due to a fault, this is an expected timing limitation that requires either a faster scan, a latching field device, or a high-speed input feature.

Task 6 — Wire It, Verify It (Inputs/Outputs + NO/NC)

Wire a pushbutton to a digital input. Before writing any logic against it, open the I/O status/monitor view and confirm the input bit toggles when the button is physically pressed.

Solution / Answer Key

Pass criteria: student demonstrates the bit changing state live in the monitor view (screenshot or instructor sign-off) *before* any rung is written. This enforces the rule: never assume a wire is correct without confirming it in software first.

Task 7 — E-Stop Fault Analysis (Inputs/Outputs + NO/NC)

Complete the table: for each wiring/actuation combination, state the resulting logic level and whether the machine stops safely.

Solution / Answer Key

NC + wire cut/unplugged -> reads logic 0 -> MACHINE STOPS (same signature as pressed) NC + actuated (pressed) -> reads logic 0 -> MACHINE STOPS (intended) NO + wire cut/unplugged -> reads logic 0 -> MACHINE KEEPS RUNNING (fault undetected!) NO + actuated (pressed) -> reads logic 1 or 0 depending on logic sense -> unreliable / not fail-safe

Conclusion: NC wiring is mandated for E-stops because a wiring fault and an intentional press are electrically indistinguishable — both are safe. NO wiring lets a real fault hide undetected.

Task 8 — Rung Translation Drill (Ladder Logic)

Translate to both ladder sketch and Structured Text: "Lamp on when either Sensor A or Sensor B is triggered, but not when the Guard Door is open."

Solution / Answer Key

Ladder: |--[SensorA]-----+----[/ GuardDoor]----- (Lamp)--| |--[SensorB]-----+
ST: Lamp := (SensorA OR SensorB) AND NOT GuardDoor;

Task 9 — Tag Naming Audit (Ladder Logic / Tag Naming)

Given a rung written entirely in raw addresses (e.g. %I0.0, %I0.1, %Q0.1), rewrite it using tags per the class naming convention, and explain in one sentence why a future maintainer would prefer the tagged version.

Solution / Answer Key

Raw: Coil(%Q0.1) := %I0.0 AND %I0.1; Tagged: Motor_Run := Start_PB AND Safety_OK;

Model answer: %I0.3 tells a future maintainer nothing about what's physically wired there six months later, while *Start_PB* and *Safety_OK* communicate intent immediately, saving debugging time without needing to cross-reference the I/O map.

Task 10 — AND/OR Project (Graded) (Ladder Logic)

Build the two-pushbutton lamp exercise: an AND version and an OR version. Tags only — raw addresses are not accepted.

Solution / Answer Key

AND version: Lamp := PB1 AND PB2; OR version: Lamp := PB1 OR PB2;

Grading check: both pushbuttons must be tagged (e.g. PB1, PB2), not %I0.0/%I0.1. Verify AND requires both pressed simultaneously; OR requires either.

Task 11 — Predict-Before-Build (Day Project — Seal-In)

Before touching software: sketch the seal-in rung on paper and answer — "what happens if the Motor_Run contact is left out?"

Solution / Answer Key

Expected prediction: without the parallel Motor_Run seal-in contact, the motor runs *only while Start_PB is physically held*. There is nothing keeping the rung true once Start_PB releases, because the seal-in contact is what allows the rung to "hold itself in" on subsequent scans.

Task 12 — Build & Debug (Day Project — Seal-In)

Wire Start (NO) / Stop (NC) / Motor output. Implement the seal-in rung and demonstrate: (a) motor stays on after Start is released, (b) Stop immediately kills it regardless of Start's state.

Solution / Answer Key

```
Ladder: |--[ Start_PB ]-----+-----[/ Stop_PB ]----- ( Motor_Run )-- | | |--[ Motor_Run ]---+ (seal-in contact, parallel with Start_PB)
```

```
ST: Motor_Run := (Start_PB OR Motor_Run) AND NOT Stop_PB;
```

Pass criteria: (a) releasing Start_PB does not turn off Motor_Run — the Motor_Run contact from the previous scan keeps the OR branch true. (b) pressing Stop_PB (NC opens, reads 0) forces the AND to false immediately, overriding both Start_PB and the seal-in contact.

Task 13 — Forward/Reverse Interlock (Day Project — Stretch)

For early finishers: implement Fwd_Run / Rev_Run with mutual exclusion. Answer: "If both pushbuttons are pressed on the exact same scan, what determines which direction wins?"

Solution / Answer Key

```
Fwd_Run := (Fwd_PB OR Fwd_Run) AND NOT Stop_PB AND NOT Rev_Run; Rev_Run := (Rev_PB OR Rev_Run) AND NOT Stop_PB AND NOT Fwd_Run;
```

Model answer: Program execution order within the scan determines the winner. Ladder logic is solved top-to-bottom in a single scan — whichever rung is evaluated first (and whose output goes true first) locks out the other on the very next scan, because each direction's rung includes a NOT of the other direction's own output. The order is determined by the rung sequence in the program, not by which button was physically pressed "first" in real time.

End of Day 1 Task Set — Total 13 tasks covering all Day 1 concept blocks.